Relacionamentos UML e implementação Java

Da modelagem estrutural ao código orientado a objetos

Associação: 1 × 1, 1 × N e N × M

Classe associativa: matrícula

Herança e polimorfismo

Agregação e composição

Como transformar o diagrama em código

Por que modelar relacionamentos?

Um sistema possui objetos que colaboram e compartilham informações.

O Diagrama de Classes documenta estrutura, atributos, operações, associações e restrições.

As linhas não viram automaticamente código correto: as regras do domínio precisam ser implementadas.

Quatro tipos de relacionamento

Associação — vínculo estrutural entre objetos.

Generalização — uma classe especializa outra (é um).

Agregação — todo-parte com independência das partes.

Composição — todo-parte com pertencimento exclusivo.

Como interpretar multiplicidades?

1 = exatamente um; 0..1 = zero ou um.

0..* = zero ou muitos; 1..* = um ou muitos.

A multiplicidade junto a B indica quantos objetos B se relacionam a um objeto A.



Associação 1 × 1 — Pessoa e Passaporte

Exemplo didático: cada pessoa tem um passaporte e vice-versa



Associação bidirecional: cada classe guarda uma referência à outra.

Unicidade e existência precisam ser garantidas por validações.

Código Java — associação 1 × 1

```
class Pessoa {
    private Passaporte passaporte;
    Pessoa(Passaporte p) {
        this.passaporte = p;
        p.definirTitular(this);
    }
}
class Passaporte {
    private Pessoa titular;
    void definirTitular(Pessoa p) { titular = p; }
}
```

Uma referência singular em cada direção não garante, sozinha, exclusividade.

Associação unidirecional × bidirecional

```
// Unidirecional: Pessoa conhece Passaporte
class Pessoa { private Passaporte passaporte; }

// Bidirecional: os dois objetos se conhecem
class Pessoa { private Passaporte passaporte; }
class Passaporte { private Pessoa titular; }
```

Navegabilidade e multiplicidade são decisões diferentes.

Associação 1 × N — Departamento e Funcionário



Um departamento tem vários funcionários; cada funcionário tem um departamento.

0..* permite departamento vazio; 1..* exigiria pelo menos um.

Código Java — associação 1 × N

```
class Departamento {
    private List<Funcionario> funcionarios =
        new ArrayList<>();
    void adicionar(Funcionario f) {
        if (f.getDepartamento() != null)
            throw new IllegalStateException();
        funcionarios.add(f);
        f.definirDepartamento(this);
    }
}
class Funcionario {
    private Departamento departamento;
}
```

Coleções e multiplicidades

1 → referência obrigatória (validar ausência de null).

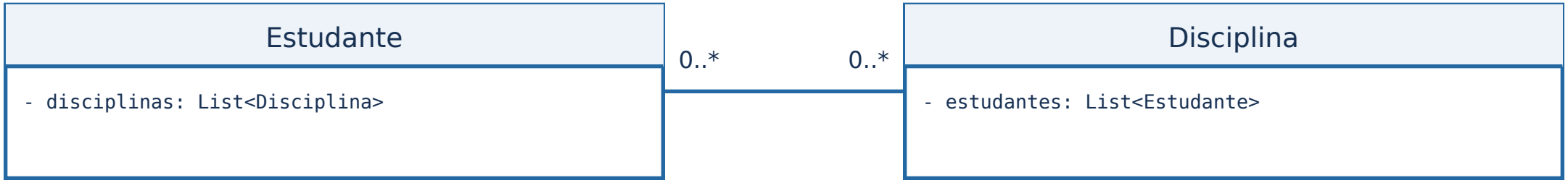
0..1 → referência opcional.

0..* → List<T>, implementada com ArrayList<T>.

1..* → List<T> + validação de quantidade mínima.

List e ArrayList não garantem invariantes UML.

Associação N × M — Estudante e Disciplina



Um estudante cursa várias disciplinas.

Uma disciplina recebe vários estudantes.

Cada extremidade pode utilizar uma List.

Código Java — associação N × M

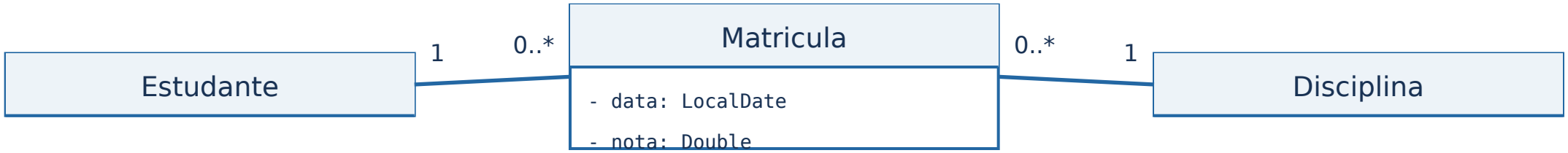
```
class Estudante {
    private List<Disciplina> disciplinas =
        new ArrayList<>();
    void matricular(Disciplina d) {
        if (!disciplinas.contains(d)) {
            disciplinas.add(d);
            d.adicionar(this);
        }
    }
}
class Disciplina {
    private List<Estudante> estudantes =
        new ArrayList<>();
    void adicionar(Estudante e) {
        if (!estudantes.contains(e)) estudantes.add(e);
    }
}
```

Quando criar uma classe associativa?

A matrícula possui data, nota e situação.

Esses dados pertencem à relação estudante–disciplina, não a uma das extremidades.

Uma classe Matrícula permite atributos, operações e regras próprias.



Código Java — classe associativa

```
class Matricula {
    private Estudante estudante;
    private Disciplina disciplina;
    private LocalDate data;
    private Double notaFinal;
    private String situacao;

    Matricula(Estudante e, Disciplina d) {
        estudante = e;
        disciplina = d;
        data = LocalDate.now();
    }
}
```

Herança — Gerente é um Funcionário



Triângulo vazio aponta para a superclasse.

Herança expressa especialização, não apenas reaproveitamento de código.

Código Java — herança

```
class Funcionario {
    protected double salario;
    double calcularPagamento() { return salario; }
}
class Gerente extends Funcionario {
    private double bonus;
    @Override
    double calcularPagamento() {
        return salario + bonus;
    }
}
```

Polimorfismo e substituição

```
Funcionario f = new Gerente();
double pagamento = f.calcularPagamento();

// A implementação invocada é a de Gerente.
```

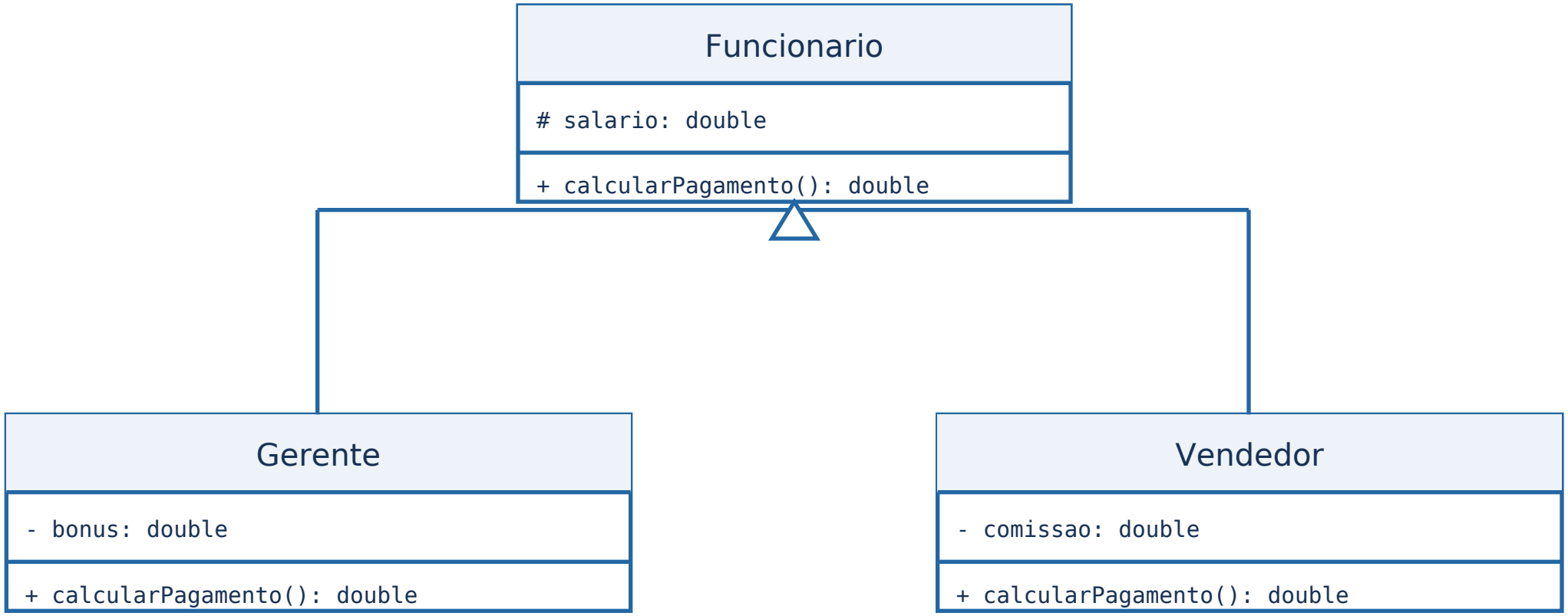
extends define especialização; @Override redefine comportamento.

A subclasse deve respeitar o contrato da superclasse.

Não use herança somente para compartilhar atributos.

Polimorfismo na UML: onde ele aparece?

Polimorfismo é um comportamento do modelo, não um novo tipo de linha UML.



Triângulo vazio = generalização; métodos iguais, implementações diferentes.

Polimorfismo no código: sobrescrita

O mesmo método possui implementações diferentes nas subclasses.

```
class Funcionario {
    protected double salario;
    Funcionario(double salario) { this.salario = salario; }
    public double calcularPagamento() { return salario; }
}
class Gerente extends Funcionario {
    private double bonus;
    Gerente(double s, double b) { super(s); bonus = b; }
    @Override
    public double calcularPagamento() { return salario + bonus; }
}
```

Polimorfismo: outro subtipo e chamada dinâmica

```
class Vendedor extends Funcionario {
    private double comissao;
    Vendedor(double s, double c) { super(s); comissao = c; }
    @Override
    public double calcularPagamento() { return salario + comissao; }
}
Funcionario a = new Gerente(6000, 1500);
Funcionario b = new Vendedor(3000, 700);
System.out.println(a.calcularPagamento()); // 7500.0
System.out.println(b.calcularPagamento()); // 3700.0
```

Polimorfismo com List e ArrayList

Uma única coleção armazena objetos de subclasses diferentes.

```
List<Funcionario> equipe = new ArrayList<>();
equipe.add(new Funcionario(3000));
equipe.add(new Gerente(6000, 1500));
equipe.add(new Vendedor(3000, 700));

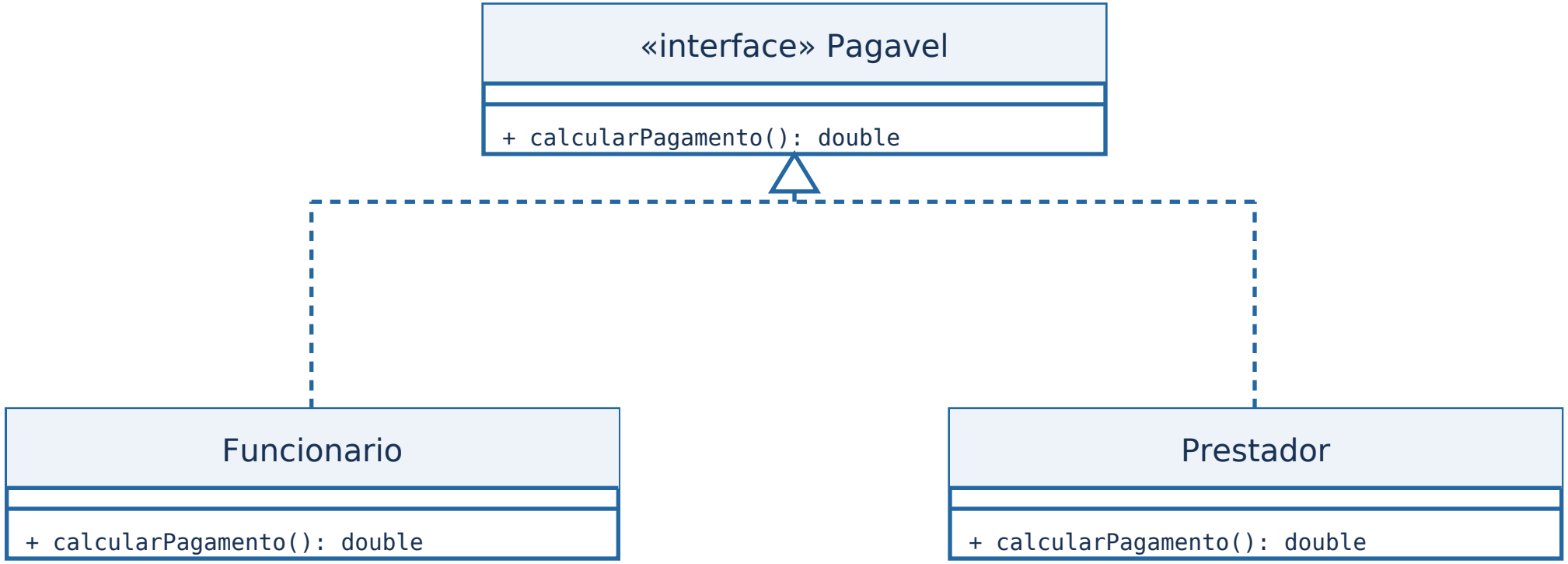
for (Funcionario funcionario : equipe) {
    System.out.println(funcionario.calcularPagamento());
}
// Saída: 3000.0, 7500.0, 3700.0
```

A JVM seleciona a implementação pelo tipo real de cada objeto.

Não é necessário testar se o objeto é Gerente ou Vendedor.

Polimorfismo: interface UML e implementação

Alternativa: classes diferentes implementam o mesmo contrato.



Na UML: realização = linha tracejada com triângulo vazio.

Em Java: implements Pagavel; chamadas usam a interface.

Código Java: polimorfismo por interface

```
interface Pagavel {
    double calcularPagamento();
}
class Funcionario implements Pagavel {
    public double calcularPagamento() { return 3000; }
}
class Prestador implements Pagavel {
    public double calcularPagamento() { return 1200; }
}
List<Pagavel> pagamentos = new ArrayList<>();
pagamentos.add(new Funcionario());
pagamentos.add(new Prestador());
for (Pagavel p : pagamentos) System.out.println(p.calcularPagamento());
```

Duas formas frequentemente chamadas de polimorfismo

Sobrescrita (override) e sobrecarga (overload) não são a mesma coisa.

Sobrescrita: mesmo método, implementações diferentes em subclasses.

- Escolha da implementação: tipo real do objeto, em execução.

Sobrecarga: mesmo nome, listas de parâmetros diferentes.

- Escolha da assinatura: argumentos e tipos conhecidos na compilação.

Terminologia: sobrescrita → polimorfismo de subtipo; sobrecarga → ad hoc.

A sobrecarga nem sempre é chamada de polimorfismo na mesma acepção.

Sobrescrita na UML: generalização e operação

A UML não usa uma linha especial para “polimorfismo”.



Cachorro herda de Animal e redefine emitirSom().

O triângulo vazio aponta para Animal (superclasse).

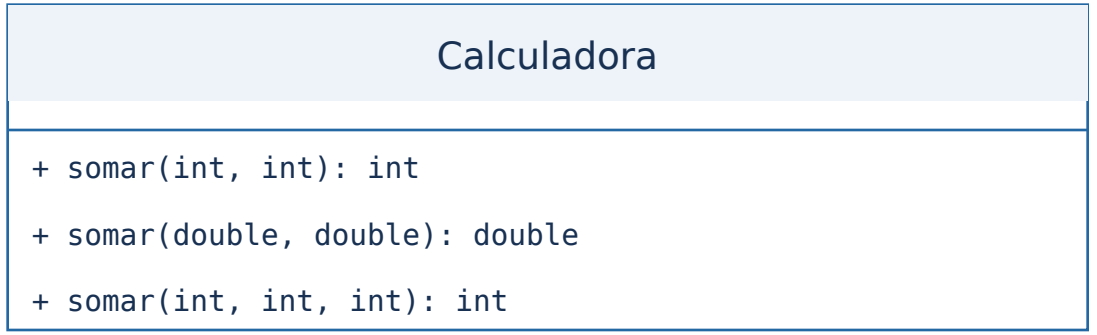
Sobrescrita em Java: despacho dinâmico

```
class Animal {
    public void emitirSom() { System.out.println("Som"); }
}
class Cachorro extends Animal {
    @Override
    public void emitirSom() { System.out.println("Au au"); }
}
Animal a = new Cachorro();
a.emitirSom();  // Au au
```

A variável é Animal, mas o objeto real é Cachorro.

Sobrecarga na UML: assinaturas distintas

Não exige herança nem generalização.



Mesma classe e mesmo nome: somar.

O número ou o tipo dos parâmetros distingue cada operação.

Sobrecarga em Java: seleção na compilação

```
class Calculadora {
    int somar(int a, int b) { return a + b; }
    double somar(double a, double b) { return a + b; }
    int somar(int a, int b, int c) { return a+b+c; }
}
Calculadora calc = new Calculadora();
calc.somar(2, 3);       // versão int, int
calc.somar(2.5, 3.5);   // versão double, double
calc.somar(2, 3, 4);    // versão com 3 parâmetros
```

Apenas mudar o tipo de retorno NÃO caracteriza sobrecarga.

Sobrescrita × sobrecarga: comparação

Critério	Sobrescrita	Sobrecarga
Assinatura	Mesma	Parâmetros diferentes
Herança	Método herdado	Não necessária
Escolha	Em execução	Na compilação
UML	Generalização + operações	Operações na mesma classe
Java	@Override	Mesmo nome, parâmetros distintos

Ambas permitem usar o mesmo nome de operação, mas por mecanismos diferentes.

Exercício: identifique a técnica

```
class Animal { void som() { } }
class Gato extends Animal {
    @Override void som() { System.out.println("Miau"); }
}
class Impressora {
    void imprimir(String texto) { }
    void imprimir(int numero) { }
}
```

1. Onde há sobrescrita? Como representá-la em UML?

2. Onde há sobrecarga? Como representá-la em UML?

3. Qual é resolvida pelo tipo real do objeto?

Agregação — Time e Jogador



Losango vazio no todo: Time.

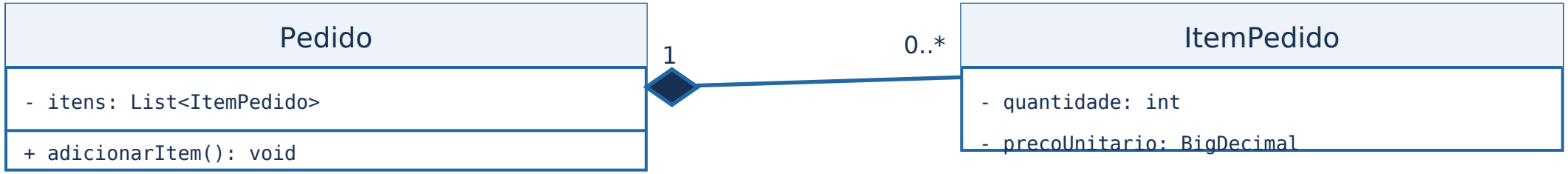
Jogadores têm identidade independente e podem continuar existindo fora do time.

Java não possui palavra-chave para agregação.

Código Java — agregação

```
class Jogador {
    private String nome;
    Jogador(String nome) { this.nome = nome; }
}
class Time {
    private List<Jogador> jogadores =
        new ArrayList<>();
    void adicionar(Jogador j) { jogadores.add(j); }
}
Jogador j = new Jogador("Pedro");
Time t = new Time();
t.adicionar(j);
```

Composição — Pedido e ItemPedido



Losango preenchido no todo: Pedido.

ItemPedido pertence exclusivamente ao pedido.

O pedido controla criação, acesso e alterações dos itens.

Código Java — composição

```
class Pedido {
    private final List<ItemPedido> itens =
        new ArrayList<>();
    void adicionarItem(Produto p, int qtd) {
        itens.add(new ItemPedido(p, qtd));
    }
    List<ItemPedido> getItens() {
        return List.copyOf(itens);
    }
}
class ItemPedido {
    ItemPedido(Produto p, int qtd) { /* valida */ }
}
```

Composição: o que o código precisa garantir?

Um item não pode pertencer simultaneamente a dois pedidos.

O pedido controla o vínculo e o ciclo de vida de seus itens.

Não exponha a coleção mutável diretamente.

Excluir um objeto Java não exclui automaticamente registros do banco.

Pedido em rascunho pode ter 0 itens; pedido confirmado pode exigir 1..*.

Comparação dos relacionamentos

Associação: vínculo entre objetos; referência ou coleção.

Agregação: todo-parte com independência das partes; losango vazio.

Composição: todo-parte com exclusividade; losango preenchido.

Herança: relação 'é um'; triângulo vazio aponta para a classe geral.

List<T> e new, isoladamente, não definem o tipo UML.

Da UML para Java: cinco etapas

1. Identificar classes → class Pedido, class Cliente...

2. Identificar atributos → private BigDecimal preco;

3. Identificar operações → calcularTotal().

4. Identificar relacionamentos → referência, coleção, extends.

5. Implementar invariantes → validações e consistência bidirecional.

Exemplo integrado — sistema de vendas



Cliente–Pedido: associação 1 × N.

Pedido–ItemPedido: composição 1 × N.

ItemPedido–Produto: associação N × 1.

Exercício prático — modelar e implementar

1. Crie Cliente, Produto, Pedido e ItemPedido.
2. Defina atributos, operações e multiplicidades.
3. Use composição entre Pedido e ItemPedido.
4. Implemente subtotal e total com BigDecimal.
5. Impeça confirmar pedidos sem itens.
6. Justifique cada associação e restrição no código.

Questões para discussão

Uma coleção sempre significa agregação? Não.

Uma referência singular garante 1 × 1? Não.

Todo N × M exige classe associativa? Não.

Quando usar classe associativa? Quando o vínculo possui dados/regras.

Herança e composição podem ser substitutas? Às vezes, mas expressam relações distintas.

Síntese — o modelo precisa orientar o código

UML documenta estrutura, multiplicidade e responsabilidades.

Java implementa referências, coleções, especialização e validações.

Associações bidirecionais exigem sincronização.

Composição exige preservar pertencimento e ciclo de vida.

Boa modelagem = relações coerentes com o domínio e invariantes verificáveis.